

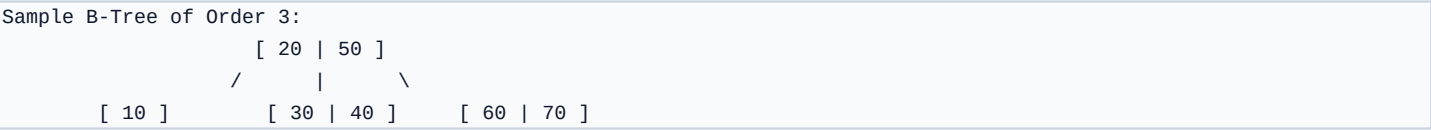
Introduction to B-Tree & B+ Tree -- Data Structures

Definition: A B-Tree is a self-balancing multi-way search tree of order m where each node can contain multiple keys and more than two children, designed specifically to optimize disk read/write block accesses in database indexing and file systems.

1. Detailed Technical Explanation

1. Properties of a B-Tree of Order `m`:

1. Every node has at most m children.
2. Every leaf node contains pointers to other leaf nodes, making them doubly linked.



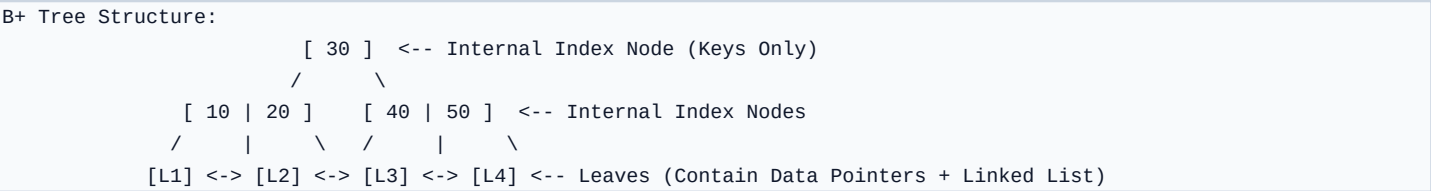
2. B-Tree vs B+ Tree Comparison

B-TREE:

- Keys AND satellite record data pointers are stored in BOTH internal and leaf nodes.

B+ TREE:

- Internal nodes store ONLY routing search keys.
- ALL satellite data records/pointers reside ONLY in LEAF nodes.
- All leaf nodes are linked together as a SLL / DLL (Doubly Linked List) for sequential range scans!



Detailed Comparison Table:

Feature	B-Tree	B+ Tree
Data Storage	Data stored in internal & leaf nodes.	Data stored ONLY in leaf nodes.
Search Efficiency	Search may finish early at internal no	Search always goes down to leaf nodes
Range Queries	Inefficient (requires in-order tree tr	**Extremely Fast** (traverse sequentia
Node Capacity	Fewer keys per disk block due to data	More keys per block (higher fanout, sm
Applications	File systems, database core storage.	Relational Database Indexing (MySQL In

3. Core Concepts & Memory Keywords

- * Order m: Maximum number of children a node can have.
- * Fanout: Number of pointers a node contains.

4. Must-Write Points for Exams

* B-Trees and B+ Trees reduce disk I/O by fitting large numbers of keys inside a single disk page block (e.g., 4KB or 8KB).

* B+

5. Quick Recall Flow

Multi-Way Search Tree -> B-Tree (Data in all nodes) vs B+ Tree (Data only in linked leaves + High Fanout for DB Indexing)